

A Practical Introduction to Machine Learning in Python

Day 2 - Tuesday Morning

»From text to features: Natural Language Processing«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 16, 2025

Today

Bottom-up vs. top-down

Where we left off yesterday

Basic string operations

A cleaner BOW representation

Better tokenization

Stopword removal

Stemming and lemmatization

ngrams

The order of preprocessing steps

How further?

Bottom-up vs. top-down

Automated content analysis can be either **bottom-up** (inductive, explorative, pattern recognition, ...) or **top-down** (deductive, based on a-priori developed rules, ...). Or in between.

The ACA toolbox

	Methodological approach		
	Counting and Dictionary	Supervised Machine Learning	Unsupervised Machine Learning
Typical research interests and content features	visibility analysis sentiment analysis subjectivity analysis	frames topics gender bias	frames topics
Common statistical procedures	string comparisons counting	support vector machines naive Bayes	principal component analysis cluster analysis latent dirichlet allocation semantic network analysis



Boumans and Trilling, 2016

Bottom-up vs. top-down

Bottom-up

- Count most frequently occurring words
- Maybe better: Count combinations of words $\Rightarrow$ Which words co-occur together?

We *don't* specify what to look for in advance

Bottom-up vs. top-down

Bottom-up

- Count most frequently occurring words
- Maybe better: Count combinations of words $\Rightarrow$ Which words co-occur together?

We *don't* specify what to look for in advance

Top-down

- Count frequencies of pre-defined words
- Maybe better: patterns instead of words

We *do* specify what to look for in advance

A simple bottom-up approach

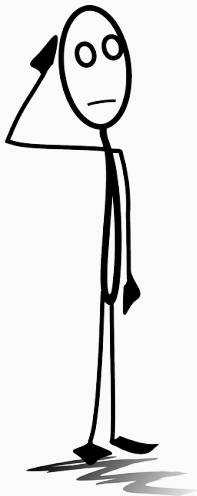
```
1 from collections import Counter
2
3 texts = ["I really really really love him, I do", "I hate him"]
4
5 for t in texts:
6     print(Counter(t.split()).most_common(3))
```

```
1 [('really', 3), ('I', 2), ('love', 1)]
2 [('I', 1), ('hate', 1), ('him', 1)]
```


A simple top-down approach

```
1 texts = ["I really really really love him, I do", "I hate him"]
2 features = ['really', 'love', 'hate']
3
4 for t in texts:
5     print(f"\nAnalyzing '{t}':")
6     for f in features:
7         print(f"{f} occurs {t.count(f)} times")
```

```
1 Analyzing 'I really really really love him, I do':
2 really occurs 3 times
3 love occurs 1 times
4 hate occurs 0 times
5
6 Analyzing 'I hate him':
7 really occurs 0 times
8 love occurs 0 times
9 hate occurs 1 times
```



When would you use which approach?

Some considerations

- Both can have a place in your workflow (e.g., bottom-up as first exploratory step)
- You have a clear theoretical expectation? Bottom-up makes little sense.
- But in any case: you need to transform your text into something "countable".

Some considerations

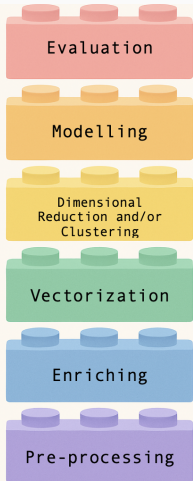
- Both can have a place in your workflow (e.g., bottom-up as first exploratory step)
- You have a clear theoretical expectation? Bottom-up makes little sense.
- But in any case: you need to transform your text into something "countable".

Some considerations

- Both can have a place in your workflow (e.g., bottom-up as first exploratory step)
- You have a clear theoretical expectation? Bottom-up makes little sense.
- But in any case: you need to transform your text into something “countable”.

Where we left off yesterday

Lego pipeline



how to assess the output of your model

how to train a model to describe (UML)
or to predict (SML)

how to group features or cases

how to turn text → numbers

how to label text data

how to tidy up text data

Problems

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs “biking”
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Four fixes in (nearly) 1 line

All together: tf-idf, smart tokenization, explicit stopword removal, pruning

```
1 myvectorizer = TfidfVectorizer(tokenizer = TreebankWordTokenizer().
    tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```

Today: customize your preprocessing

Remember the black box nature of more modern models. Already in the one-liner above, many choices are *hidden*.

Today: customize your preprocessing

Remember the black box nature of more modern models. Already in the one-liner above, many choices are *hidden*.

⇒ Example of AI coding to highlight why understanding is important: use AI wisely!

Basic string operations

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

Working with strings

1. string methods that every string has (`"hello".upper()`)
2. functions that take a string as input (`len("hello")`)
3. pandas column string methods
(`df["somecolumn"].str.upper()`)
4. applying string methods or functions to a pandas column
(`df["somecolumn"].apply(len)` or
`df["somecolumn"].apply(lambda x: x.upper())`)

For today, we assume that our data are a list of strings – adapt accordingly for pandas.

An example

```
1 # probably read from text file(s) instead, you learned that already...
2 data = [ "I <b>really</b> liked this movie! It was great. ", " What an
      awful movie", "Awesome!!!"]
3
4 data_stripped = [e.strip() for e in data]
5 data_lower = [e.lower() for e in data_stripped]
6 data_clean = [e.replace("<b>","").replace("</b>","") for e in data_lower
      ]
7
8 # or, more efficient, in one single step:
9 data_clean2 = [e.strip().lower().replace("<b>","").replace("</b>","")
      for e in data]
```

Another example

```
10 from string import punctuation
11
12 # punctuation is just the string '!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
13
14 text = "This is a test! Let's get rid (of) punct&"
15
16 # we make a list of each character in the text but only if it is not
17 # a punctuation sign. The, we join the elements of the list directly
18 # to each other without anything between it ("")
19 cleantext = "".join([c for c in text if c not in punctuation])
```

Combine both

```
20 from string import punctuation
21
22 def strip_punctuation(text):
23     return "".join([c for c in text if c not in punctuation])
24
25 data_clean3 = [strip_punctuation(e).strip().lower()\
26     .replace("<b>", "").replace("</b>", "") for e in data]
```

The toolbox at a glance

Slicing

`mystring[2:5]` to get the characters with indices 2,3,4

String methods

- `.lower()` returns lowercased string
- `.strip()` returns string without whitespace at beginning and end
- `.find("bla")` returns index of position of substring "bla" or -1 if not found
- `.replace("a", "b")` returns string with "a" replaced by "b"
- `.count("bla")` counts how often substring "bla" occurs
- `.isdigit()` true if only numbers

Use tab completion for more!

Basic string operations

A cleaner BOW representation

Room for improvement

tokenization How do we (best) split a sentence into tokens
(terms, words)?

pruning How can we remove unnecessary words?

lemmatization How can we make sure that slight variations of the
same word are not counted differently?

OK, good enough, perfect?

`.split()`

- space → new word
- no further processing whatsoever
- thus, only works well if we do a preprocessing ourselves (e.g., remove punctuation)

```
1 docs = ["This is a text", "I haven't seen John's derring-do. Second  
    sentence!"]  
2 tokens = [d.split() for d in docs]
```

```
1 [['This', 'is', 'a', 'text'], ['I', "haven't", 'seen', "John's", 'derring-do.', 'Second', '  
    sentence!']]
```

OK, good enough, perfect?

Tokenizers from the NLTK package

- multiple improved tokenizers that can be used instead of `.split()`
- e.g., Treebank tokenizer:
 - split standard contractions ("don't")
 - deals with punctuation
 - BUT: Assumes lists of *sentences*.
- Solution: Build an own (combined) tokenizer (next slide)!

OK, good enough, perfect?

```
28 import nltk
29 import regex
30
31 class MyTokenizer:
32     def tokenize(self, text):
33         tokenizer = nltk.tokenize.TreebankWordTokenizer()
34         result = []
35         word = r"\p{letter}"
36         for sent in nltk.sent_tokenize(text):
37             tokens = tokenizer.tokenize(sent)
38             tokens = [t for t in tokens
39                     if regex.search(word, t)]
40             result += tokens
41         return result
42
43 mytokenizer = MyTokenizer()
44 tokens = [mytokenizer.tokenize(d) for d in docs]
```

```
1 [['This', 'is', 'a', 'text'], ['I', 'have', "n't", 'seen', 'John', "'s", 'derring-do', 'Second',
    'sentence']]
```

Stopword removal

What are stopwords?

- Very frequent words with little inherent meaning
- the, a, he, she, ...
- context-dependent: if you are interested in gender, he and she are no stopwords.
- Many existing lists as basis

Stopword removal: What and why?

Why remove stopwords?

- If we want to identify key terms (e.g., by means of a word count), we are not interested in them
- If we want to calculate document similarity, it might be inflated
- If we want to make a word co-occurrence graph, irrelevant information will dominate the picture

Stopword removal

```
1 from nltk.corpus import stopwords
2 mystopwords = stopwords.words("english")
3 mystopwords.extend(["test", "this"])
4
5 def tokenize_clean(s, stoplist):
6     cleantokens = []
7     for w in TreebankWordTokenizer().tokenize(s):
8         if w.lower() not in stoplist:
9             cleantokens.append(w)
10    return cleantokens
11
12 tokens = [tokenize_clean(d, mystopwords) for d in docs]
```

```
1 [['text'], ["n't", 'seen', 'John', 'derring-do.', 'Second', 'sentence', '!']]
```

You can do more!

For instance, in line 8, you could add an `and` statement to also exclude punctuation.

Basic string operations

Stemming and lemmatization

NLP: What and why?

Why do stemming?

- Because we do not want to distinguish between smoke, smoked, smoking, . . .
- Typical preprocessing step (like stopword removal)

Stemming and lemmatization

- Stemming: reduce words to its stem by removing last part (drinking → drink)
- Lemmatization: find word that you would need to look up in a dictionary (drinking → drink, but also went → go)
- stemming is simpler than lemmatization
- lemmatization often better

Example below: tokenization and lemmatization with spacy in one go:

```
1 import spacy
2 nlp = spacy.load('en') # potentially you need to install the language
  model first
3 lemmatized_tokens = [[token.lemma_ for token in nlp(doc)] for doc in
  docs]

1 [['this', 'be', 'a', 'text'], ['-PRON-', 'have', 'not', 'see', 'John', "s", 'derring', '-', 'do
  ', '!', 'second', 'sentence', '!']]
```

Stemming and lemmatization

- Stemming: reduce words to its stem by removing last part (drinking → drink)
- Lemmatization: find word that you would need to look up in a dictionary (drinking → drink, but also went → go)
- stemming is simpler than lemmatization
- lemmatization often better

Example below: tokenization and lemmatization with spacy in one go:

```
1 import spacy
2 nlp = spacy.load('en') # potentially you need to install the language
  model first
3 lemmatized_tokens = [[token.lemma_ for token in nlp(doc)] for doc in
  docs]
```

```
1 [['this', 'be', 'a', 'text'], ['-PRON-', 'have', 'not', 'see', 'John', "'s", 'derring', '-', 'do',
  ', .', 'second', 'sentence', '!']]
```

Basic string operations

ngrams

Instead of just looking at single words (unigrams), we can also use adjacent words (bigrams).

ngrams

```
1 import nltk
2 texts = ['This is the first text text text first', 'And another text
           yeah yeah']
3 texts_bigrams = [["_".join(tup) for tup in nltk.ngrams(t.split(),2)] for
                    t in texts]
4 print(texts_bigrams)
```

```
[['This_is', 'is_the', 'the_first', 'first_text',
  'text_text', 'text_text', 'text_first'],
 ['And_another', 'another_text', 'text_yeah',
  'yeah_yeah']]
```

Typically, we would combine both. *What do you think? Why is this useful? (and what may be drawbacks?)*

ngrams

```
1 import nltk
2 texts = ['This is the first text text text first', 'And another text
    yeah yeah']
3 texts_bigrams = [["_".join(tup) for tup in nltk.ngrams(t.split(),2)] for
    t in texts]
4 print(texts_bigrams)
```

```
[['This_is', 'is_the', 'the_first', 'first_text',
'text_text', 'text_text', 'text_first'],
['And_another', 'another_text', 'text_yeah',
'yeah_yeah']]
```

Typically, we would combine both. **What do you think? Why is this useful? (and what may be drawbacks?)**

Basic string operations

The order of preprocessing steps

Option 1

Preprocessing only through Vectorizer

“Just use CountVectorizer or TfidfVectorizer with the appropriate options.”

- pro: No double work, efficient if your main goal is a sparse matrix (for ML?) anyway
- con: you cannot “see” the preprocessed texts

Option 2

Extensive preprocessing without Vectorizer

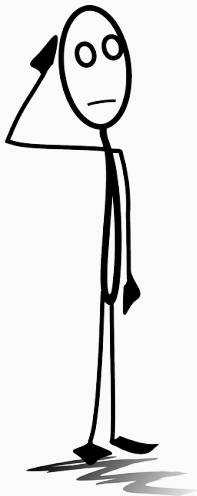
“Remove stopwords, punctuation etc. and store in a string with spaces”

```
1 cleaneddocs = [" ".join(re.findall(r"\w\w+", d)).lower() for d in docs]
2 cleaneddocswithoutstopwords = [" ".join([w for w in d.split() if w not
    in mystopwords]) for d in cleaneddocs]
```

```
1 ['this is text', 'haven seen john derring do second sentence']
2 ['text', 'seen john derring second sentence']
```

Yes, this list comprehension looks scary – you can make a more elaborate for loop instead

- pro: you can read (and store!) the preprocessed docs
- pro: even the most stupid vectorizer (or wordcloud tool) can split the resulting string later on
- con: potentially double work (for you *and* the computer)



How would you do it?

Sometimes, you might go for Option 2 because

- you like to inspect a sample of the documents
- you can re-use the cleaned docs irrespective of the Vectorizer

But at other times, you might opt of Option 1 instead because

- you want to systematically compare the effect of different choices in a machine learning pipeline (then you can simply vary the vectorizer instead of the data)
- you want to use techniques that are geared towards little or no preprocessing (deep learning)

Basic string operations

How further?

Main takeaway

- It matters how you transform your text into numbers (“vectorization”).
- Preprocessing matters, be able to make informed choices.
- Keep this in mind when we will discuss Machine Learning!
- Once you vectorized your texts, you can do all kinds of calculations (random example: get the cosine similarity between two texts)

More NLP

n -grams Consider using n -grams instead of unigrams

collocations *ngrams* that appear more frequently than expected

POS-tagging grammatical function ("part-of-speech") of tokens

NER named entity recognition (persons, organizations, locations)

General approach

Test on a single string, then make a for loop or list comprehension!

Own functions

If it gets more complex, you can write your own function and then use it in the list comprehension:

```
1 def mycleanup(t):
2     # do sth with string t here, create new string t2
3     return t2
4
5 results = [mycleanup(t) for t in allmytexts]
```


Pandas string methods as alternative

If you select column with strings from a pandas dataframe, pandas offers a collection of string methods (via `.str.`) that largely mirror standard Python string methods:

```
1 df['newcolumnwithresults'] = df['columnwithtext'].str.count("bla")
```

To pandas or not to pandas for text?

Partly a matter of taste.

Not-too-large dataset with a lot of extra columns? Advanced statistical analysis planned? Sounds like pandas.

It's mainly a lot of text? Wanna do some machine learning later on anyway? It's large and (potentially) messy? Doesn't sound like pandas is a good idea.

